

Welches KI-Modell passt wirklich?

Benchmarking frei verfügbarer Sprachmodelle für Audience-Response-Systeme am Beispiel von frag.jetzt

Elvis-Elijah Giani

18. Februar 2026

Zusammenfassung

Große Sprachmodelle sind heute leicht zugänglich – doch welches Modell eignet sich wirklich für den Einsatz in einem Audience-Response-System? Gerade in der Hochschullehre entscheiden Antwortqualität, Latenz, Kosten und Verlässlichkeit darüber, ob ein KI-Assistent sinnvoll unterstützt oder eher stört. Dieser Artikel zeigt, wie sich frei verfügbare KI-Sprachmodelle systematisch vergleichen lassen: nachvollziehbar, fair und messbar durch einen strukturierten Benchmarking-Ansatz. Am Beispiel von *frag.jetzt* wird erläutert, welche Use-Cases relevant sind, wie RAG-Szenarien bewertet werden können und welche Rolle Leistungsmetriken wie Latenz und Kosten in der Praxis spielen.

1 Wie wählt man aus vielen frei verfügbaren KI-Sprachmodellen das passende Modell für ein Audience-Response-System?

Audience-Response-Systeme (ARS) erweitern Lehrveranstaltungen um eine digitale Interaktionsebene. Teilnehmende können in Echtzeit Fragen stellen, Feedback geben oder Themen priorisieren. Didaktisch entsteht dadurch ein klarer Mehrwert, weil sich mehr Personen beteiligen und Lehrende schneller erkennen, wo Verständnisprobleme auftreten (Worms, 2026). Hinzu kommt ein praktischer Effekt: Die Möglichkeit anonymer Beiträge senkt insbesondere in großen Gruppen die Hemmschwelle deutlich (frag.jetzt, 2025).

Sobald ein ARS KI-generierte Antworten integriert, wird die Wahl des Sprachmodells zu einer technischen und zugleich organisatorischen Weichenstellung. Im Live-Betrieb treten vor allem drei Aspekte in den Vordergrund: die Qualität der Antworten, also ihre fachliche Richtigkeit, Verständlichkeit und Nützlichkeit, die Latenz und Stabilität des Systems sowie die wirtschaftlichen Rahmenbedingungen mit Blick auf Kosten, Limits und Skalierbarkeit. Als Beispielkontext dient hier *frag.jetzt*. Dort werden KI-Assistenten für Lernräume beschrieben, einschließlich Prompt-Anpassung, Mehrsprachigkeit, Quellenangaben und Moderationsoptionen (frag.jetzt, 2026). Ergänzend existiert das sogenannte „Fragen-Radar“, das zentrale Begriffe aus vielen Beiträgen extrahiert und als gewichtete Wortwolke zur Navigation darstellt (Educa,

2026; frag.jetzt, 2025). Aktuelle Dokumente beschreiben frag.jetzt zudem als System, das sich vom klassischen ARS in Richtung eines KI-gestützten „Personal Learning Environment“ weiterentwickelt (frag.jetzt, 2025; A. / frag.jetzt, 2025).

1.1 Was bedeutet „passendes Modell“ im ARS-Kontext?

Viele Modellvergleiche im Internet beantworten vor allem die Frage, welches System allgemein als besonders leistungsfähig gilt. Für ein ARS ist diese Perspektive zu unspezifisch. Ein Modell ist hier dann passend, wenn es bei typischen ARS-Aufgaben zuverlässig gute Ergebnisse liefert und sich zugleich unter realen Betriebsbedingungen verantworten lässt, also in Bezug auf Performance, Ausfallverhalten, Datenschutz und Kosten.

Gerade im Bereich der Moderation zeigt sich, weshalb ein einzelner Gesamtscore wenig aussagekräftig ist. In der frag.jetzt-Analyse wird eine Designentscheidung beschrieben, bei der das Moderationssystem bewusst auf hohe Sensitivität, also hohen Recall, optimiert wurde. Ziel ist es, problematische Inhalte möglichst selten zu übersehen, auch wenn dadurch mehr Fehlalarme entstehen können (frag.jetzt, 2025). Welche Lösung als geeignet gilt, hängt daher weniger von einem Spitzenwert in einem Benchmark ab als vom akzeptablen Fehlerprofil im jeweiligen Lehrkontext.

2 Welche Wege gibt es, ein Modell einzubinden?

Die Art des Zugangs ist keine reine Implementierungsfrage, sondern beeinflusst unmittelbar, was im Betrieb gemessen und bewertet werden muss. Kostenmodelle, Ausfallverhalten, Datenschutzanforderungen und Skalierungsmöglichkeiten variieren je nach Einbindungsstrategie. Gleichzeitig orientieren sich viele Schnittstellen an ähnlichen API-Schemata, etwa im Stil von Chat-Completions oder Responses. Dadurch lässt sich ein Benchmarking-Setup häufig mit überschaubarem Anpassungsaufwand gegen unterschiedliche Backends betreiben (llama.cpp, 2026; MLC-AI, 2026; OpenAI, 2026a; OpenRouter, 2026b).

2.1 Direkte Provider-API

Bei diesem Ansatz wird ein Modell unmittelbar beim jeweiligen Anbieter genutzt. Die Infrastruktur ist in der Regel stabil und gut dokumentiert, was die Integration vereinfacht. Dem stehen Abhängigkeiten vom Provider, potenzielle Netzwerklatenz sowie feste Limits und Preismodelle gegenüber (OpenAI, 2026a).

2.2 Gateway oder Router

Ein Router bündelt verschiedene Modelle hinter einem einheitlichen Endpunkt und erleichtert so den Wechsel zwischen Anbietern. Technisch relevant ist, dass Routing-Logiken und Fallback-Mechanismen dazu führen können, dass nicht immer dasselbe Modell antwortet. Für belastbares Benchmarking sollte daher die tatsächlich verwendete Modell-ID systematisch mitprotokolliert werden. OpenRouter beschreibt Modell-Fallbacks als Verfahren, bei dem bei Fehlern oder Rate-Limits automatisch auf ein nachrangiges Modell gewechselt wird (OpenRouter, 2026a, 2026b).

2.3 Lokale oder selbst gehostete LLMs

Bei selbst betriebenen Modellen laufen Inferenz und Speicherung auf eigener Infrastruktur, etwa auf einem Server oder in einer virtuellen Maschine. Dieser Ansatz ist im Hinblick auf Datenhoheit attraktiv, bringt jedoch zusätzlichen Aufwand für Betrieb, Monitoring und Lasttests mit sich. Werkzeuge wie Ollama erleichtern den Einstieg (Ollama, 2026). Für die Integration ist relevant, dass Server-Stacks rund um `llama.cpp` OpenAI-kompatible Endpunkte bereitstellen und sich damit gut in ein einheitliches Test-Setup einfügen (llama.cpp, 2026).

2.4 Browserbasierte LLMs

Hier läuft das Modell direkt im Browser, etwa mithilfe von WebGPU. Der Vorteil liegt darin, dass keine serverseitige Inferenz erforderlich ist. Gleichzeitig verschieben sich die Bewertungskriterien auf Aspekte wie Geräteabhängigkeit, Initialisierungszeit und Speicherbedarf (MLC-AI, 2026).

Zugang	Typische Stärken	Typische Risiken
Provider-API	stabil, wenig Ops	Limits/Preise, Abhängigkeit
Router	flexibel, schneller Wechsel	Routing/Fallback, Policy
Lokal	Kontrolle, Datenhoheit	Betrieb, HW, Skalierung
Browser	kein Server nötig	Geräte, Startzeit, Memory

Tabelle 1: Zugangswege und Trade-offs (vereinfachte Übersicht).

3 Was muss ein Modell im ARS wirklich können?

Ein belastbares Benchmarking orientiert sich an den konkreten Aufgaben im System und nicht an allgemeinen Leaderboards. Für *frag.jetzt* lassen sich aus Dokumentation und Analyse mehrere Aufgabenfelder ableiten, darunter Live-Q&A, didaktische Erklärungen, dokumentbasierte Fragen im Sinne von Retrieval-Augmented Generation (RAG), die Strukturierung vieler Beiträge etwa über Radar- oder Clustering-Mechanismen sowie Moderation (Educa, 2026; frag.jetzt, 2025, 2026; A. /. frag.jetzt, 2025).

Das Handout macht zudem deutlich, dass KI in *frag.jetzt* nicht auf das bloße Beantworten von Fragen beschränkt ist. Genannt werden unter anderem das Generieren von Übungs- und Prüfungsaufgaben, die Erstellung von Lernmaterialien wie Anki-Karten, Zusammenfassungen sowie Quiz- und Rallye-Formate (A. /. frag.jetzt, 2025). Für ein sinnvolles Benchmarking ist das entscheidend, da solche Szenarien deutlich näher an realen Lehr- und Lernsituationen liegen als abstrakte Wissensfragen ohne didaktischen Kontext.

Use-Case	Beispiel-Aufgabe	Messsignal (Auszug)
Live-Q&A	kurze, korrekte Antwort	Rubrik + p95-Latenz
Erklärung	Schrittfolge + Mini-Beispiel	Rubrik (Didaktik, Struktur)
RAG-Q&A	Frage zum Skript/Upload	Recall@k + Faithfulness (RAGAS)
Radar/Tags	Stichwörter, Cluster, Top-Themen	Konsistenz + Navigationsnutzen
Lernmaterial	Quiz-/Karteikarten aus Skript	Validität + Format-Checks
Moderation	unangemessene Inhalte erkennen	Recall/Precision, Schwellenwerte

Tabelle 2: Typische ARS-Use-Cases und passende Messsignale (kompakt).

4 Ein technischer Blick auf frag.jetzt: Warum das für Benchmarking relevant ist

Der Benchmarking-Ansatz wird konkreter, wenn man betrachtet, wie ein ARS KI tatsächlich integriert. In der Analyse zu *frag.jetzt* wird eine modulare Architektur hervorgehoben, die anbieterunabhängig ausgelegt ist und damit das Risiko eines Vendor-Lock-in reduziert (frag.jetzt, 2025). Als zentrale Bausteine werden unter anderem LangChain und LangGraph genannt, die zur Orchestrierung von KI-Workflows sowie zur Einbindung unterschiedlicher Modelle, Datenquellen und Werkzeuge dienen (frag.jetzt, 2025; A. /. frag.jetzt, 2025).

frag.jetzt, 2025). Für das Benchmarking folgt daraus, dass nicht nur die Modellwahl relevant ist. Auch Parameter wie Chunking-Strategien, Embedding-Modelle, Prompt-Design oder Moderationslogik stellen eigenständige Stellschrauben dar, die getrennt untersucht werden können und das Gesamtergebnis maßgeblich beeinflussen.

4.1 RAG in frag.jetzt: mehr als nur „Dokument hochladen“

Retrieval-Augmented Generation wird in frag.jetzt als zentrale Strategie zur Reduktion von Halluzinationen beschrieben. Antworten sollen sich auf eine vom Lehrenden bereitgestellte Wissensbasis stützen, etwa Skripte oder Fachartikel (frag.jetzt, 2025; A. / frag.jetzt, 2025). Der zugrunde liegende Ablauf umfasst mehrere aufeinander aufbauende Schritte: Zunächst werden Dokumente hochgeladen und in Chunks zerlegt, anschließend in Vektorrepräsentationen überführt und indexiert. Darauf folgt das Retrieval relevanter Textabschnitte, die schließlich als Kontext in die Antwortgenerierung einfließen (frag.jetzt, 2025).

Bemerkenswert ist, dass unterschiedliche Chunking-Strategien systematisch gegeneinander evaluiert werden, darunter splitterbasiertes und semantisches Chunking (frag.jetzt, 2025). Für das Benchmarking bedeutet das, dass nicht allein das Sprachmodell bewertet werden darf. Die gesamte Pipeline, vom Chunking über das Retrieval bis zur finalen Antwort, bildet die relevante Einheit.

4.2 Radar und Keywords: von klassischen NLP-Pipelines zu Chat-Modellen

Für die Strukturierung vieler eingehender Fragen beschreibt frag.jetzt eine Entwicklung, die auch über dieses System hinaus typisch ist. Anstelle klassischer NLP-Pipelines, die beispielsweise nur Substantive extrahieren, wird ein Chat-Modell eingesetzt, das zusätzlich benannte Entitäten und numerische Angaben identifizieren kann. Die extrahierten Schlüsselbegriffe werden anschließend als gewichtete Wortwolke im „Fragen-Radar“ visualisiert (Educa, 2026; frag.jetzt, 2021, 2025).

Im Benchmarking steht hier weniger die stilistische Qualität einzelner Antworten im Vordergrund, sondern die inhaltliche Konsistenz und der praktische Nutzen. Entscheidend ist, ob die generierten Stichwörter die Diskussion angemessen abbilden und in großen Sessions tatsächlich Orientierung bieten.

4.3 Moderation: warum Recall im Lehrkontext entscheidend sein kann

Die Analyse zu frag.jetzt beschreibt eine mehrstufige Moderationspipeline, die von tokenbasierter Profanitätserkennung über Sentiment-Analyse bis hin zu um-

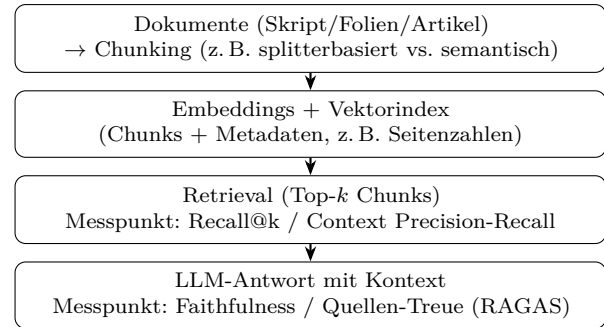


Abbildung 1: RAG-Pipeline im ARS-Kontext: Retrieval und Antwortgenerierung sollten getrennt evaluiert werden.

fassenderen Klassifikationsmodellen reicht (frag.jetzt, 2025). Hervorgehoben wird die bewusste Optimierung auf hohe Sensitivität, also hohen Recall, um problematische Inhalte möglichst selten zu übersehen (frag.jetzt, 2025).

Für ein aussagekräftiges Benchmarking bedeutet das, dass neben einer globalen Accuracy insbesondere Precision- und Recall-Werte, gewählte Schwellenwerte sowie die Rate an Fehlalarmen im jeweiligen Lehrkontext berücksichtigt werden müssen.

5 Benchmarking-Strategien: Baseline + Use-Case + Messaufbau

In der Praxis empfiehlt sich eine Kombination mehrerer Ebenen. Standard-Benchmarks liefern einen groben Orientierungsrahmen, die eigentliche Auswahl sollte jedoch auf systemnahen Use-Case-Tests beruhen.

5.1 Standard-Benchmarks als Baseline

Frameworks wie HELM ermöglichen eine transparente und reproduzierbare Einordnung von Modellen über unterschiedliche Szenarien und Metriken hinweg (CRFM, 2026; Liang, Bommasani et al., 2022). Ergänzend existieren öffentliche Leaderboards, die offene Modelle anhand fest definierter Benchmark-Suites vergleichen, etwa das Open LLM Leaderboard (Face, 2026). Solche Übersichten helfen, grundlegende Stärken und Schwächen zu erkennen. Für Entscheidungen im ARS-Kontext sind sie jedoch nur bedingt ausreichend, da spezifische Anforderungen wie RAG-Treue, Moderationsverhalten, Keyword-Extraktion oder didaktische Erklärqualität dort meist nur indirekt erfasst werden.

5.2 Use-Case-Benchmark als Kern

Im Zentrum steht daher ein eigenes, kuratiertes Testset, beispielsweise mit 60 bis 120 Prompts, verteilt

über die zuvor definierten Aufgabenfelder aus Tabelle 2. Entscheidend ist die Vergleichbarkeit der Bedingungen. Prompt-Vorlagen, Systemvorgaben und Sampling-Parameter wie Temperatur, Top-p und maximale Tokenanzahl sollten identisch sein. Wiederholte Durchläufe können zusätzlich helfen, Varianz sichtbar zu machen. Um eine realistische ARS-Nähe zu erreichen, sollte ein Teil der Prompts aus authentischen, anonymisierten Fragen stammen. Ergänzend sind gezielte Kontrollfälle sinnvoll, etwa Fragen, deren Antwort bewusst nicht im bereitgestellten Dokumentenkorpus enthalten ist.

5.3 Reproduzierbarkeit als zentrale Voraussetzung

In der Praxis scheitern Modellvergleiche häufig weniger an fehlenden Metriken als an unzureichender Dokumentation. Ohne Angaben zur Modellversion, zum genauen Prompt, zu Parametern oder zum tatsächlich abgerufenen Kontext im RAG-Setup sind Ergebnisse kaum nachvollziehbar. Evaluationsframeworks unterstützen dabei, solche Informationen strukturiert zu erfassen. OpenAI beschreibt mit *Evals* ein Framework, mit dem sich eigene Tests definieren und systematisch gegen Modelle oder komplette LLM-Systeme ausführen lassen (OpenAI, 2026b, 2026c). Für standardisierte Benchmarks ist zudem der *lm-evaluation-harness* verbreitet, der als technische Grundlage vieler Benchmark-Läufe dient (EleutherAI, 2026).

5.4 Qualitätsbewertung: Rubrik und Paarvergleiche

Bei offenen Textantworten reichen binäre Metriken wie „richtig“ oder „falsch“ meist nicht aus. Bewährt hat sich eine kompakte Bewertungsrubrik mit einer Punkteskala von 0 bis 5 über wenige stabile Kriterien, darunter Korrektheit, Relevanz, Verständlichkeit und Struktur, didaktischer Mehrwert sowie Transparenz im Umgang mit Unsicherheit. Im RAG-Kontext kommt zusätzlich die Treue zum bereitgestellten Kontext hinzu.

Für direkte Modellvergleiche sind paarweise Bewertungen oft robuster als absolute Punktzahlen. In der Chatbot Arena werden Antworten zweier Modelle gegeneinander gestellt und daraus Ratings abgeleitet (LMSYS, 2023; Zheng et al., 2023a, 2023b). Dieses Prinzip lässt sich im kleineren Maßstab auf ein ARS-Testset übertragen, indem identische Fragen blind mit zwei Modellantworten bewertet werden.

5.5 RAG gesondert bewerten: Retrieval und Antwort

Die Evaluation von RAG-Systemen umfasst zwei Ebenen. Ist das Retrieval unzureichend, kann selbst ein leistungsfähiges Modell keine kontexttreue Antwort erzeugen. Umgekehrt nützt ein guter Retriever wenig, wenn das Modell den gelieferten Kontext nicht angemessen berücksichtigt. RAGAS schlägt hierfür unter

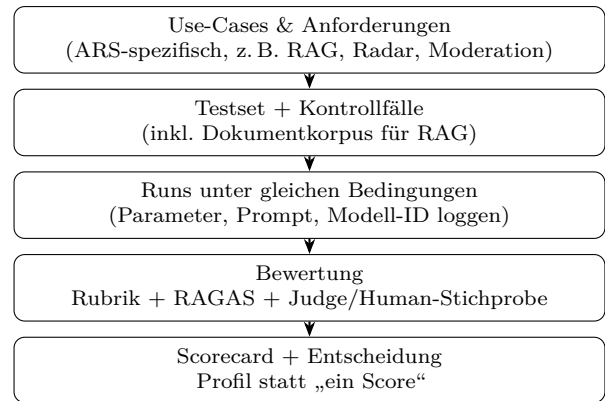


Abbildung 2: Benchmarking im ARS-Kontext: aus Use-Cases werden reproduzierbare Tests und eine begründbare Auswahl.

anderem Metriken wie Faithfulness, Answer Relevance sowie Context Precision und Recall vor (Es et al., 2023, 2024). Ergänzend sind klassische Retrievalmetriken wie Recall@k oder Mean Reciprocal Rank sinnvoll, da sie schnell sichtbar machen, ob relevante Chunks unter den Top-*k* Treffern erscheinen (Pinecone, 2026; Weaviate, 2024).

6 Harte Metriken: Latenz, Durchsatz, Stabilität, Kosten

Für ein ARS ist die inhaltliche Qualität zentral, doch im Live-Betrieb prägt vor allem die wahrgenommene Reaktionsgeschwindigkeit die Akzeptanz. Latenz sollte daher nicht nur als Mittelwert ausgewiesen werden, sondern zusätzlich über Perzentile wie p50 und p95. Ergänzend ist die *Time to First Token* (TTFT) relevant. Sie misst die Zeitspanne von der Anfrage bis zum ersten ausgegebenen Token und korreliert stark mit der subjektiv empfundenen Responsivität (Anyscale, 2026; NVIDIA, 2026). Bei längeren Antworten spielt zudem die Inter-Token-Latenz beziehungsweise die Generationsrate in Tokens pro Sekunde eine Rolle, da sie bestimmt, wie schnell der vollständige Text erscheint (Anyscale, 2026).

Bei Lasttests genügt es nicht, lediglich den maximalen Durchsatz zu betrachten. Entscheidend ist, welcher Anteil der Antworten innerhalb einer definierten Ziel-Latenz bleibt. In Performance-Dokumentationen wird hierfür der Begriff *Goodput* verwendet, also jener Durchsatz, der unter vorgegebenen Latenzanforderungen tatsächlich nutzbar ist (BentoML, 2026). Stabilität lässt sich zusätzlich über Fehlerquoten, Timeout-Raten und das Verhalten bei parallelen Anfragen charakterisieren.

Viele APIs rechnen tokenbasiert ab. Für belastbare Vergleiche sollte daher der Tokenverbrauch pro Use-Case getrennt nach Input und Output protokolliert und in Kosten pro 100 oder 1000 Anfragen überführt

werden. Im ARS-Kontext ist dies besonders relevant, da RAG-Setups den Prompt um zusätzlichen Kontext erweitern und dadurch sowohl Latenz als auch Tokenverbrauch beeinflussen (frag.jetzt, 2025).

7 Ergebnisdarstellung: Scorecard als Entscheidungsbasis

Um die Auswahl transparent zu machen, bietet sich eine kompakte Scorecard an, die Qualitäts- und Systemmetriken zusammenführt. Maßgeblich ist nicht ein einzelner Gesamtwert, sondern das Profil eines Modells: Ein System kann didaktisch besonders überzeugend sein, ein anderes durch geringe Latenz oder hohe RAG-Zuverlässigkeit. In produktiven Architekturen ist darüber hinaus relevant, dass Router explizite Fallback-Mechanismen unterstützen, sodass bei Limits oder Fehlern automatisch auf ein alternatives Modell gewechselt werden kann (OpenRouter, 2026a).

Modell	Qual.	RAG	p95	Kosten
A	4.4	4.1	2.8s	m
B	3.9	3.2	1.6s	n
C	4.2	4.5	3.4s	h

Tabelle 3: Beispielhafte Scorecard (m = mittel, n = niedrig, h = hoch).

8 Fazit

Ein Audience-Response-System benötigt keine abstrakt „beste“ KI, sondern ein Modell, das im eigenen Anwendungskontext zuverlässig arbeitet. Entscheidend sind verständliche Live-Antworten, kontexttreue RAG-Ergebnisse, tragfähige Moderation und ein betrieblich beherrschbares Systemverhalten. Ein strukturierter Benchmarking-Prozess kombiniert deshalb allgemeine Baselines wie HELM oder öffentliche Leaderboards mit ARS-spezifischen Use-Cases, darunter Live-Q&A, didaktische Erklärungen, RAG, Radar-gestützte Strukturierung, Lernmaterial-Generierung und Moderation. Werden diese qualitativen Prüfungen um harte Metriken wie TTFT, p95-Latenz, Stabilität und Kosten ergänzt, entsteht eine fundierte Entscheidungsgrundlage anstelle einer rein intuitiven Auswahl.

Quellen und weiterführende Literatur

Anyscale. (2026). Understand LLM latency and throughput metrics (TTFT, ITL, Throughput) [Zugriff: 18. Februar 2026].

BentoML. (2026). Key metrics for LLM inference (Latency, Throughput, Goodput) [Zugriff: 18. Februar 2026].

CRFM, S. (2026). HELM: Holistic Evaluation of Language Models [Zugriff: 18. Februar 2026].

Educa. (2026). frag.jetzt (Educa): Fragen-Radar und Stichwort-Extraktion [Zugriff: 18. Februar 2026].

EleutherAI. (2026). lm-evaluation-harness [Zugriff: 18. Februar 2026].

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAs: Automated Evaluation of Retrieval Augmented Generation [Preprint]. *arXiv preprint arXiv:2309.15217*. <https://arxiv.org/abs/2309.15217>

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAs: Automated Evaluation of Retrieval Augmented Generation. *Proceedings of EACL (Demo)*. <https://aclanthology.org/2024.eacl-demo.16/>

Face, H. (2026). Open LLM Leaderboard [Zugriff: 18. Februar 2026].

frag.jetzt. (2021). Topic Cloud [Zugriff: 18. Februar 2026].

frag.jetzt. (2025). frag.jetzt Analyse für Hochschullehre [Zugriff: 18. Februar 2026].

frag.jetzt. (2026). frag.jetzt: Q&A Rooms & AI Assistants [Zugriff: 18. Februar 2026].

frag.jetzt, A. /. (2025). Handout zum KI-Workshop frag.jetzt (PDF) [Zugriff: 18. Februar 2026].

Liang, P., Bommasani, R., et al. (2022). Holistic Evaluation of Language Models [Preprint]. *arXiv preprint arXiv:2211.09110*. <https://arxiv.org/abs/2211.09110>

llama.cpp. (2026). Server README: OpenAI-compatible endpoints [Zugriff: 18. Februar 2026].

LMSYS. (2023). Chatbot Arena: Benchmarking LLMs in the Wild with Elo Ratings [Zugriff: 18. Februar 2026].

MLC-AI. (2026). WebLLM [Zugriff: 18. Februar 2026].

NVIDIA. (2026). Metrics — NVIDIA NIM LLM Benchmarking (TTFT, Throughput, etc.) [Zugriff: 18. Februar 2026].

Ollama. (2026). Ollama [Zugriff: 18. Februar 2026].

OpenAI. (2026a). Create chat completion [Zugriff: 18. Februar 2026].

OpenAI. (2026b). Getting Started with OpenAI Evals (Cookbook) [Zugriff: 18. Februar 2026].

OpenAI. (2026c). openai/evals: Framework for evaluating LLMs and LLM systems [Zugriff: 18. Februar 2026].

OpenRouter. (2026a). Model Fallbacks (Automatic Failover) [Zugriff: 18. Februar 2026].

OpenRouter. (2026b). Quickstart Guide [Zugriff: 18. Februar 2026].

Pinecone. (2026). RAG Evaluation: Don't let customers tell you first [Zugriff: 18. Februar 2026].

Weaviate. (2024). Evaluation Metrics for Search and Recommendation Systems (Precision@k, Re-

-
- call@k, MRR, nDCG) [Zugriff: 18. Februar 2026].
- Worms, H. (2026). Audience Response System [Zugriff: 18. Februar 2026].
- Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023a). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Preprint]. *arXiv preprint arXiv:2306.05685*. <https://arxiv.org/abs/2306.05685>
- Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023b). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html

Hinweis zur KI-Nutzung: Bei der Erstellung dieses Artikels wurden KI-Tools (ChatGPT) zur Unterstützung bei Strukturierung, Recherche und sprachlicher Ausarbeitung eingesetzt. Die inhaltliche Verantwortung liegt beim Autor.